

研究ストーリー

S2620154 吉田 将衛

2026 年 5 月 8 日

一言で言う（elevator pitch）

ドキュメント記述に頼らず、代替が存在する非推奨化候補をコード構造・使用パターン・Git 履歴から自動検出する手法を提案する（動機: 代替追加 → @Deprecated 付与まで平均 **8 年** のラグが存在する）

確定済みスコープ

項目	決定内容	根拠・備考
エコシステム	Java	Sawant et al. の先行研究群に準拠
ターゲットベニュー	SCAM	ソースコード解析・MSR 寄りの貢献が評価される
ステークホルダー	提供者 (API producer) 向け	Sawant 2019「提供者ポリシーが消費者行動を動かす」と整合
フレーム	手法提案 (doc-free 検出)	Javadoc に書いてあれば開発者はすでに知っている → ドキュメント非依存シグナルで価値を出す

1 背景

破壊的変更と非推奨化の役割

ライブラリの提供者 (API producer) が既存のメソッドを削除・変更するとき、消費者 (API consumer) のコードはコンパイルエラーや実行時エラーを起こす破壊的変更 (breaking change) となる [1, 5]。そのため提供者は削除の前に @Deprecated アノテーションを付与し、消費者が代替機能へ移行するまでの猶予を設けることが慣習となっている [4]。

@Deprecated を付与する最大の動機は「代替機能の導入 (NF)」であり、Sawant et al. [4] の調査 (Java 4 ライブラリ・374 件) で全体の **37%** (138 件) を占める。すなわち、提供者が代替機能 B を追加した時点が旧機能 A を非推奨化すべき自然なタイミングとなっている。

非推奨化が遅延・放置される構造的問題

しかし、代替機能が追加された後も旧機能への @Deprecated 付与が長期間遅延するケースが広く存在することが、本研究のパイロット分析で示唆される（詳細は次節）。

Brito et al. [1] が破壊的変更を導入した開発者 56 名に行ったインタビューでは、@Deprecated を付与しなかった 17 名への追加質問において、「**非推奨化後のメンテナン**

スコスト増加（44%）」が最多回答であった。同論文は「@Deprecated を付与すると削除するまで旧版と新版の両方を並行して更新し続けるコストが生じる」ことを理由として挙げており、開発者は必要性を認識しつつもコストを理由に後回しにしていることが示されている。しかし本研究のパイロット分析では、代替追加から @Deprecated 付与までの平均ラグが 8 年（2,904 日）であることが観察されており（次節）、先送りによって並行維持期間が長期化し、回避しようとしたコストが逆に増大していることを示唆する。

さらに、Sawant [2] は提供者の非推奨化ポリシー（削除積極性等）が消費者の移行行動を直接左右することを大規模分析（25,357 プロジェクト）で実証した。提供者が非推奨化を後回しにすることで、消費者の移行も連鎖的に遅延するという構造的な問題が生じる。

なぜドキュメント非依存の検出が必要か

非推奨化を進めたい提供者にとって、プロジェクト全体のどのメソッドが候補になりうるかを特定する作業は手動の調査に頼らざるを得ない。技術的負債の議論では、問題が定量的に可視化されることで初めて優先付けの対象となる **[TODO: 引用が必要（技術的負債の可視化と優先付けに関する先行研究）]**。先行研究のスコープは付与済みの @Deprecated に限られているため、付与前の候補を体系的に特定するツールが存在せず、探索コストが高いままになっている。しかし根本的な困難がある：B が A の代替であるという関係は、提供者が @Deprecated を付与する時点で初めて宣言されるものである。付与前の段階では B が追加されていてもその代替関係はコード上の事実として確定しておらず、Javadoc に代替が明記されているのはパイロット分析でも 15.9% に過ぎない（詳細は次節）。

先行研究（Sawant [4, 3, 2]）はいずれも付与済みの @Deprecated を起点とするため、「付与前の候補発見」はスコープ外であった。したがって、提供者の宣言を待たずに候補を提示するには、コード構造・使用パターン変化・Git 履歴という客観的シグナルから代替関係を推定するアプローチが必要となる。本研究は OSS の Git 履歴を用いた遡及的分析により、観測可能なシグナルが将来の非推奨化と相関するかを検証する。

2 Motivation Example

（2026-05-04 計測済み）

指標	値	補足
has_replacement_pct	15.9%	@Deprecated 全件中 Javadoc に代替メソッド記載あり
alt_found_pct	93.9%	記載ありのうち Git から代替を自動発見できた割合
nf_first_pct	12.6%	@Deprecated 全件中「代替が先に追加」のケース
nf_first / has_replacement	≈ 79%	Javadoc 代替記載ありの約 8 割が NF 先
lead_time_mean (NF 先のみ)	2,904 日（約 8 年）	代替追加 → @Deprecated 付与までの平均ラグ

→ 観察: Javadoc に書いてあるケースですら 8 年放置される。Javadoc にすら書かれていないケースはさらに多数存在するはず。

3 アプローチ（提案手法）

ドキュメント非依存の deprecated 候補検出

1. 動機付け分析（Javadoc 分析 OK）：Motivation Example の数値で「8 年放置が常態」を示す

2. シグナル選定（ドキュメント禁止）：以下の特徴量を設計

- 使用パターン変化: caller が method A → method B へ段階的移行（Git 履歴）
- コード構造類似: 同クラス内での類似シグネチャ・重複パラメータ（AST 解析）
- コミット共変: method B 追加コミットで method A の caller が同時変更
- 命名パターン: `getX` / `getXNew`、`doY` / `doYV2` など規則的ペア

3. 検出: 上記特徴量で非推奨化候補を特定するモデル/ルール

4. 評価: 後に `@Deprecated` が付いたメソッドを、付与前の時点で検出できるか（precision / recall）

4 Research Questions

ギャップから RQ への導出

§ 背景で示した通り、本研究のギャップは「付与前の deprecated 候補を体系的に見出す手段がない」ことにある。この困難の中核は、 B が A の代替であるという関係はコード上の事実として直接観測できず、提供者が `@Deprecated` を付与する時点で初めて宣言されるという点である。パイロット分析が示すように、Javadoc に代替が明記されているのは全体の 15.9% に過ぎず、残り 84.1% については代替関係が文書化されていない。

そこで本研究は、宣言を待たずに代替関係を推定するために、Javadoc に頼らずコード構造・使用パターン・Git 履歴から読み取れる手がかり（以下、コードと履歴に残る手がかり）を用いて、段階的に二つの問いに答える。

両 RQ 共通の時間軸 本研究では、ある観測時点 T_{obs} におけるプロジェクト開始から T_{obs} までのコード状態と Git 履歴のみを手法の入力とする。すなわち手法は T_{obs} より未来の情報を一切参照しない。RQ1 では T_{obs} を変数として掃引し、リードタイム ($T_{\text{dep}} - T_{\text{obs}}$) と検出精度の関係を測る。RQ2 では T_{obs} を最新コミットに固定して実プロジェクトに適用する。手法そのものは両者で共通であり、差は T_{obs} の置き方と答え合わせが可能か否かのみである（表 1）。

段階	RQ1（リードタイム特性の検証）	RQ2（実プロジェクト適用）
観測時点 T_{obs}	各事例ごとに $[T_B + \epsilon, T_{\text{dep}} - \epsilon]$ を掃引。 T_B は代替 B の追加時点、 T_{dep} は <code>@Deprecated</code> 付与時点	リポジトリの最新コミット時点
特徴の観測窓	プロジェクト開始 ~ T_{obs}	プロジェクト開始 ~ T_{obs}
手法適用時点	各 T_{obs} (A はまだ未付与)	T_{obs} (未付与メソッド全体が対象)
答え合わせ	T_{dep} 時点の Javadoc 記載 B と照合	右打ち切りデータに対する生存分析 (Cox 比例ハザード) で C-index・時間依存 AUC を評価

Table 1: RQ1/RQ2 の時間軸

4.1 RQ1: コードと履歴に残る手がかりは、`@Deprecated` 付与のどれだけ前まで遡って代替関係を再現できるか？

時間軸を一級の変数として扱う理由 本研究の動機は「代替 B の追加から `@Deprecated` 付与までの平均ラグが約 8 年」という事実にある。仮に手法が付与直前 $T_{\text{dep}} - \epsilon$ の状態

でしか機能しないなら、それは付与寸前の状態を確認しているに過ぎず、早期検出には貢献しない。逆に B の追加直後 $T_B + \epsilon$ で機能すれば、**8 年のラグを 0 に近づける**ことが原理的に可能となる。そこで RQ1 では観測時点 T_{obs} を $[T_B + \epsilon, T_{\text{dep}} - \epsilon]$ の範囲で掃引し、**リードタイム vs 検出精度の特性曲線**を求めることを目的とする。

母集団・入力・予測対象

- **母集団**: Java OSS プロジェクト群 **[TODO: プロジェクト選定基準と件数を S 評価設計 で定義]** から収集した事例 $(A, B, T_B, T_{\text{dep}})$ の集合。 A は @Deprecated 付与済みかつ Javadoc に代替メソッドが明記されたメソッド、 B はその代替メソッド、 T_B は B がコミットに初出現した時点、 T_{dep} は A に @Deprecated が付与された時点。パイロットでは全 @Deprecated の 14.9% ($= 15.9\% \times 93.9\%$) がこの条件を満たす
- **各事例ごとの観測時点群**: T_{obs} を以下の複数点で評価する。
 - $T_{\text{dep}} - \epsilon$ (付与直前。最も観測情報が豊富)
 - $T_{\text{dep}} - 1$ 年, $T_{\text{dep}} - 3$ 年, $T_{\text{dep}} - 5$ 年 (ラグの中盤)
 - $T_B + \epsilon$ (代替が現れた直後。理論値最速)
- **入力**: 各 T_{obs} について、プロジェクト開始から T_{obs} までのコード状態と Git 履歴のみ
- **予測対象**: 各 T_{obs} において、同一プロジェクト内のメソッド集合の中から、 T_{dep} 時点で Javadoc が指す代替メソッド B を 1 件特定する
- **答え合わせ**: Javadoc 記載の B と推定結果の一致を Precision / Recall / Recall@ k で評価。Javadoc 自体は学習・推論に一切用いず、答え合わせ専用とする

評価指標 RQ1 はランキングタスクである (正解の代替 B は各事例につき 1 件)。正解が 1 件のとき $\text{Recall}@k = \text{Hit Rate}@k$ (上位 k 件に B が入るか否かの 0/1) であり、 $\text{Precision}@k = \text{Recall}@k / k$ と等価なため Precision は省略する。

- **Recall@ k (主指標, $k \in \{1, 3, 5\}$)**: 上位 k 件に正解 B が含まれるか。 $k = 5$ を主指標とする根拠は、提供者が実際に確認できるショートリストの上限が実務上 5 件程度という前提、およびコード推薦研究での慣例 (@1/@5/@10) に従うことにある。
- **MRR (補助指標)**: $\text{MRR} = \frac{1}{|D|} \sum \frac{1}{\text{rank}(B)}$ 。Recall@ k が「入ったか否か」の 0/1 しか測れないのに対し、MRR は 2 位と 50 位を区別するため、リードタイムが伸びたときのランキング悪化の粒度を細かく追える。
- **リードタイム vs 精度曲線**: 各 T_{obs} における Recall@5 と MRR をプロットし、付与直前から T_B 直後へ遡るにつれて精度がどう悪化するかを可視化。
- **実用閾値**: ある精度水準 (例: $\text{Recall}@5 \geq 80\%$) を維持できる最大リードタイム (=「8 年中、何年前まで遡れるか」)

ベースラインと比較設計 比較群を役割の異なる 3 層に分けて設計する。

層 1—下限値の確定

- **B0: ランダム選択。** 同一プロジェクト内のメソッドから無作為に B を 1 件選ぶ。提案手法が「シグナルなし」の基準を超えることを確認する最低限の参照点 (Recall@1 $\approx 1/|M|$ となるべき)。

層 2—アブレーション (シグナル別寄与の分離)

提案手法は命名・コード構造・Git 履歴を統合して用いる。各シグナルがリードタイム別にいつ立ち上がるかを分離するために、一つずつ有効にした単独モデルを設定する。

- **B1: 命名規則のみ。** $getX/getXNew \cdot doY/doYV2$ 等の正規表現ペアで候補を生成。 T_B 直後から利用可能な最もコストの低いシグナル
- **B2: コード構造類似のみ。** AST 距離・シグネチャ類似度のみ。Git 履歴を一切参照しない
- **B3: Git 履歴のみ。** caller 段階移行・コミット共変のみ。AST 解析を行わない

B1~B3 のリードタイム vs 精度曲線を重ねることで、「どのシグナルがいつ有効になるか」を定量化する。(仮説: 命名は T_B 直後から有効・Git 移行シグナルは累積に時間を要する)

層 3—対抗手法 (LLM との対比)

- **B4: LLM ゼロショット抽出。** Javadoc を除去したメソッド本体と同一プロジェクト内の候補リストを LLM に与え、代替を 1 件選ばせる。

B4 はアブレーション (シグナル分離) ではなく、「LLM だけで十分では？」という代替アプローチとの対比である。この比較が答える問いは、「コードと履歴から計算したシグナルが LLM の暗黙知識を超える独自寄与を持つか」であり、とくに T_{obs} が T_B 直後 (Git 履歴が薄い時点) と T_{dep} 直前 (情報が豊富な時点) で逆転が起きるかが焦点となる。プロンプト設計・モデルバージョン・API コストは § 評価設計で記述する。

すべての比較群を同一の T_{obs} 掃引で評価し、リードタイム vs 精度曲線を重ねて描く。

スコープ境界と反証可能性 本手法はすべての代替関係を当ててることを目指さない。コード構造・使用パターン・Git 履歴のいずれかに痕跡を残すサブセットのみを対象とする。

反証可能な主張: 本研究の中核的主張は「ランダム選択 (B0) を有意に上回るリードタイム範囲が少なくとも 1 つ存在する」ことである。この主張は以下の形で反証される:

- $T_{obs} = T_{dep} - \epsilon$ (情報が最も豊富な時点) においてすら Recall@5 が B0 水準を上回らない場合 → 観測可能なシグナルに代替関係の情報は含まれないという否定的結論を報告する
- 有意に上回る範囲が存在しても、その最大リードタイムが 1 年未満にとどまる場合 → 「8 年のラグを縮める」という実用価値は限定的であることを認める

「リードタイムが長いほど精度が下がる」傾向自体は否定せず、悪化の度合いと、実用的な閾値 (例: Recall@5 $\geq 80\%$ を維持できる最大リードタイム) を定量化することを RQ1 の貢献とする。さらに、当てられた事例と外した事例の差を分析することで「本手法が機能する条件」 (例: caller の段階的移行が観測できる場合・命名が規則的ペアの場合 等) を明示し、適用可能な文脈を提供者に示す。

4.2 RQ2: RQ1 で同定された代替候補を共変量とする生存分析モデルは、@Deprecated 付与のハザードをどの程度の精度でランク付けできるか？

RQ1 から RQ2 への設計上の連鎖 RQ1 が答えるのは「どのメソッド A に対して代替 B がすでに存在するか」という per-pair の問いである。しかし提供者が実際に必要とするのは「プロジェクト内の未付与メソッド全体の中で、どれを先に非推奨化すべきか」という優先順位付けである。前者は代替関係の有無を明らかにするが、複数の候補が存在するときどれが急を要するかは答えない。

この問いは本質的に時間の問題であり、二値分類や回帰では扱えない。未付与メソッドのうち将来付与されるものは右打ち切りされているが、これはまさに生存分析 (survival analysis) が設計された問題設定である。そこで RQ2 は RQ1 で同定された代替候補を共変量として取り込み、時間依存共変量を含む拡張 Cox 比例ハザードモデルでハザード順ランキングを提供者に提供する。

RQ1 への依存リスクと頑健性: RQ2 の問題設定 (§ 生存分析の問題設定) は「RQ1 の手法で代替候補 B が同定された全 A 」を観察対象とするため、RQ1 の再現率が低ければ RQ2 の対象母集団も縮小する。しかし RQ2 のモデルは RQ1 の出力を共変量の一つとして扱うに過ぎず、命名・AST 距離・Git 履歴等の共変量は RQ1 とは独立に計算できる。ベースライン S4 (RQ1 結果を共変量に含めない Cox) との差分が「RQ1 の出力が RQ2 のランキング精度に与える上乗せ分」を定量化し、両 RQ の連鎖が実用的に成立することを裏付ける。

生存分析の問題設定

- 観察対象 (subject) : RQ1 の手法で代替候補 B が同定された全 A
- 時間原点 $t = 0$: T_B (代替メソッド B が初出現したコミット時点)
- イベント: A に @Deprecated が付与される
- 観察時間:
 - イベント発生例: $T_{\text{dep}} - T_B$
 - 右打ち切り例: $T_{\text{now}} - T_B$
- 時間不変共変量: 命名規則ペアか否か、 A と B の AST 距離・シグネチャ類似度 等
- 時間依存共変量: A の caller のうち B へ移行した割合、コミット共変累積回数、 B の caller 数の伸び率 等
- モデル: 拡張 Cox 比例ハザードモデル

母集団

- イベント発生群: Javadoc に代替が明記され @Deprecated が付与された 14.9% の事例 (T_{dep} が観測される)
- 右打ち切り群: 最新スナップショット時点で未付与のメソッドのうち、RQ1 の手法で代替候補 B が同定されたもの (T_{now} で打ち切り)

評価指標 RQ2 が答える問いは「ハザード順位の精度」と「予測確率の信頼性」の両面を含む。5 指標を役割別に 3 群に分けて設定する。

群 1—ランキング精度 (RQ2 の中核)

- **Concordance index (C-index)** : 無作為に選んだ 2 メソッドのうち先に非推奨化される方を正しくランクできる確率。提供者向けトリアージリストの精度に直結するため主指標とする (0.5 = ランダム、1.0 = 完全予測)
- **時間依存 AUROC (1 年・3 年・5 年)** : 「 t 年以内に付与されるか」という各地平線での二値識別能力。C-index が全時間軸を一括評価するのに対し、提供者が計画する具体的な期間 (次期リリースサイクル等) で識別できるかを測る。地平線ごとに性能プロファイルが異なれば、「短期予測は得意・長期は苦手」という実用上の限界として報告する

群 2—キャリブレーション (ランキングを確率として使えるか)

- **Brier スコア**: $\hat{S}(t|x)$ (予測生存確率) と実測値の平均二乗誤差。C-index はランキング順位しか評価せず、「ハザード 0.8」が本当に 80% の確率を意味するかは測れない。§ 応用で上位 N 件を選ぶ際の閾値設定を信頼できるものにするために必要。また S2 (ロジスティック回帰、右打ち切り無視) との Brier 差が「打ち切りを正しく扱うことでキャリブレーションが改善するか」を示す

群 3—可視化と解釈 (結果の説明可能性)

- **Kaplan-Meier 曲線**: 評価指標ではなく可視化ツール。パイロットの「平均 8 年ラグ」を単一数値ではなく生存関数の分布として描き、「ハザードの高い群 vs 低い群」の層別 KM 曲線でモデルが実際に分離できているかを視覚的に確認する
- **ハザード比**: 各共変量の効果量を表す解釈ツール。(例: 「caller 移行率が 50% 進むとハザード X 倍」) 提供者が「どのシグナルが最も非推奨化を促進するか」を理解するための Actionable Insight として提示する

ベースラインと比較設計 RQ2 の核心は「なぜ生存分析でなければならないか」と「なぜ時間依存共変量が必要か」を定量的に示すことにある。そのため、評価対象を提案手法 (時間依存共変量を含む拡張 Cox) と以下のベースラインで対比する。

- **S0: ランダムランキング**。下限値の確定 (C-index ≈ 0.5 となるべき)
- **S1: メソッド年齢ランキング**。「古い順に並べれば良いのでは?」という素朴反論への防御。パイロットの 8 年ラグから自然に導かれる仮説
- **S2: ロジスティック回帰 (主ベースライン)**。右打ち切りを単に「未付与 = 負例」として扱い、全共変量を投入。S2 と提案手法の C-index・Brier 差が、生存分析採用の必要性そのものを表す
- **S3: 時間依存共変量を除外した Cox (時間不変共変量のみ)**。S3 と提案手法の差が動的シグナル (caller 移行率等) の寄与を表す
- **S4: RQ1 結果を共変量に含めない Cox**。段階的設計 (RQ1 $\rightarrow$ RQ2) の連鎖が実用ランキングに寄与することを裏付ける

主張ラインは二段: S2 との差で「打ち切りを正しく扱う必要性」、S3 との差で「時間依存共変量の必要性」を順に示し、両者が同時に効いて初めて**拡張 Cox** という**選択**が正当化されるという論理にする。

応用: 提供者向けランキング 学習済みモデルを最新スナップショット時点に適用し、未付与メソッドを**現時点でのハザード**でランク付けする。上位 N 件は提供者がトリアージすべき候補となる。サンプル定性検証（コミット履歴・Issue・PR を手動確認）により、「妥当な候補」「誤検出」「議論済みで意図的に放置」を分類する。

スコープ境界・前提

- **選択バイアス**: イベントを生むのは過去に @Deprecated を付与した経験のある提供者のみ。一切付与しないプロジェクトは無限打ち切り扱いとなり、ベースラインハザードを下方に歪める可能性がある
- **意図的先送りの扱い**: Brito et al. [1] が報告した「コスト懸念で意図的に未付与」は競合リスク (competing risk) として扱う余地があるが、永久放置の判定は原理的に不可能なため本研究では単一イベント Cox に留める
- **時間原点の定義**: T_B を「 B の最初のコミット」とするが、テスト未整備な初期版を含む可能性があるため § 評価設計で頑健性を検証する **[TODO: T_B の定義の頑健性検証方針を § 評価設計 で詳述]**
- **比例ハザード仮定**: caller 移行効果は時間と共に変化し得るため、Schoenfeld 残差検定で仮定の妥当性を検証する

5 期待される貢献

- ✓ 代替追加 → @Deprecated のラグが平均 8 年という定量的実証 (パイロット済み)
- 複数 Java OSS プロジェクトにわたる大規模定量分析 (RQ1)
- ドキュメント非依存の特徴量設計 (コード構造・使用パターン・Git)
- アブレーション (命名／構造／履歴／LLM ゼロショット) による**手がかり別寄与のリードタイム特性**の定量化 (RQ1)
- 上記特徴量と生存分析を組み合わせた deprecated 候補ランキング手法の提案と、打ち切り無視ロジスティック・時間依存共変量なし Cox との比較による有効性検証 (RQ2)

6 差別化・位置づけ

軸	先行研究	本研究
対象	非推奨化後の消費者行動 (Sawant 2019, Cogo 2021)	非推奨化されていない放置候補の発見
シグナル	@Deprecated 解析 / 手動分析	ドキュメント非依存 (コード・使用・Git)
貢献	非推奨化の分類・影響測定	8 年放置の実証 + ドキュメント不要の検出手法
ステークホルダー	消費者 (移行コスト)	提供者 (気づいていない保守負債の可視化)

7 ストーリーの変遷

バージョン	日付	主な変更
v1	2026-04-28	初稿（雛形作成）
v2	2026-04-28	問題設定の甘いポイント整理・ステークホルダーを提供者向けに確定・スコープ（Java/SCAM）確定
v3	2026-04-28	パイロット実証の結果パターン別シナリオ（A～H）を追記
v4	2026-05-04	パイロット実証数値取得。フレームを“prediction / early detection” から “Deprecation Debt Discovery” へピボット。
v5	2026-05-04	手法提案論文として再設計。Javadoc 依存の RQ2 を廃止。ドキュメント非依存シグナル（コード・使用・Git）による候補検出を手法の核心に。

References

- [1] Aline Brito, Laerte Xavier, Andre Hora, and Marco Tulio Valente. Why and how Java developers break APIs. In *Proceedings of the 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2017.
- [2] Anand Ashok Sawant. *The impact of API evolution on API consumers and how this can be affected by API producers and language designers*. PhD thesis, Delft University of Technology, 2019.
- [3] Anand Ashok Sawant, Maurício Aniche, Arie van Deursen, and Alberto Bacchelli. Understanding developers’ needs on deprecation as a language feature. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pp. 561–571. ACM, 2018.
- [4] Anand Ashok Sawant, Guangzhe Huang, Gabriel Vilen, Stefan Stojkovski, and Alberto Bacchelli. Why are features deprecated? An investigation into the motivation behind deprecation. In *Proceedings of the 34th IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 13–24. IEEE, 2018.
- [5] Laerte Xavier, Andre Hora, Marco Tulio Valente, and Romain Robbes. Why do we break APIs? first answers from developers. In *Proceedings of the 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 392–396. IEEE, 2017.